

Introduction

The L1 InstructionCache holds a single line of 8 words, a hit costs 10 cycles, while a miss first checks L2 for a total of 30 cycles on an L2 hit or 370 cycles on an L2 miss. An L2 cache with 4 lines of 8 words, shared between instructions and data, costing 20 cycles on a hit and 360 cycles when a line must be filled from main memory.

Three programs of increasing memory complexity were written and run. A register only loop that sums the integers from 1 to 100 without touching memory, an array program that first stores values 1–100 at addresses 200–299 and then sums them with loads, and a linked-list program that builds 100 nodes at addresses 200–399 and traverses them to find the sum.

Results

Program	No Cache	IC Only	IC + L2	IC + L2
			(instr only)	(instr + data)
Sum 1–100 (registers)	61,510	2,980	3,000	3,000
Sum array (memory)	243,222	68,162	68,202	50,622
Sum linked list (memory)	392,704	198,494	135,834	100,334

Conclusion

Without a cache, the register-only program cost 61,510 cycles. Adding the L1 instruction cache cut that to 2,980, a 20x improvement. The memory programs benefited from adding L2 data caching. The array dropped from 68,162 to 50,622 cycles, and the linked list from 135,834 to 100,334, as repeated loads could be served from L2 instead of main memory. Across all three programs, the results confirm that cache effectiveness depends on the more predictable and sequential the access pattern, the greater the gain.